龙芯 2K300/301 系列板卡使用手册-编码器使用

龙芯 2K300/301 开源库使用说明

编码器使用篇

一、编码器模块简介

编码器（Encoder）是一种将机械位移转换为电信号的传感器，广泛应用于电机测速、位置检测、机器人控制等领域。

龙芯 2K300/301 开源库提供了基于 PWM 控制器捕获功能的编码器接口，核心细节如下：

【工作原理】

- 通过 PWM 控制器的捕获功能测量 A 相信号的脉宽/周期
- 通过 DIR 相信号（方向引脚）判断旋转方向
- 根据脉宽和编码器线数计算实时速度（转/秒）

【支持功能】

- A 相脉宽测量
- 方向检测功能
- 实时速度计算
- 多通道同时工作

【返回值说明】

- encoder_get_count() 返回的是实时速度（单位：转/秒）
- 正数表示正转，负数表示反转
- **重要提示：不是累积的脉冲计数！**

本库支持的编码器类型：带方向编码器

二、包含头文件

在使用编码器功能之前，需要在代码中包含以下头文件：

```
c++
#include "lq_reg_pwm_encoder.hpp"
```

该头文件定义了编码器相关的类、枚举类型和函数。

三、编码器数据类型定义

3.1 编码器 PWM 通道枚举 (enc_pwm_channel_t)

用于指定使用的编码器通道：

- ENC_PWM_CH0：通道 0
- ENC_PWM_CH1：通道 1
- ENC_PWM_CH2：通道 2
- ENC_PWM_CH3：通道 3
- ENC_PWM_CH_INVALID：无效通道

3.2 编码器 PWM 可选引脚枚举 (ls_enc_pwm_pin_t)

用于指定编码器脉冲输入引脚：

- ENC_PWM0_PIN64：PIN_64（复用模式 ALT1）
- ENC_PWM1_PIN65：PIN_65（复用模式 ALT1）
- ENC_PWM2_PIN66：PIN_66（复用模式 ALT1）
- ENC_PWM3_PIN67：PIN_67（复用模式 ALT1）
- ENC_PWM0_PIN86：PIN_86（复用模式 ALT2）

注意：

- 编码器需要两个引脚：脉冲引脚和方向引脚
- 脉冲引脚从上述枚举中选择
- 方向引脚需要单独指定一个普通 GPIO 输入引脚

3.3 编码器线数宏定义

```c++
#define LQ_NUM_ENCODER_LINE  (512)
```

默认编码器线数为 512 线，根据实际使用的编码器修改此值。

常见的编码器线数有：100、200、256、512、1024、2048 等。

四、ls_encoder_pwm 类接口说明

4.1 构造函数

【无参构造函数】

```c++
ls_encoder_pwm()
```

功能：创建一个未初始化的编码器对象

【有参构造函数】

```c++
ls_encoder_pwm(ls_enc_pwm_pin_t _pin, gpio_pin_t _dir)
```

功能：初始化编码器对象

参数：

- _pin：编码器脉冲引脚（从 ls_enc_pwm_pin_t 枚举中选择）
- _dir：编码器方向引脚（普通 GPIO 输入引脚）

示例：

```c++
ls_encoder_pwm encoder(ENC_PWM0_PIN64, PIN_72);
```

4.2 控制函数

【获取编码器速度】

```c++
float encoder_get_count(void)
```

功能：获取编码器实时速度（带方向）

返回：实时速度，单位为转/秒（正数表示正转，负数表示反转）

计算原理：

```c++
速度 = 时钟频率 / 脉冲周期 / 编码器线数 * 方向
```

其中方向由方向引脚电平决定（高电平=+1，低电平=-1）

注意：

- 返回值是实时速度，不是累积的脉冲计数
- 每次调用都返回当前瞬间的速度值

【重置计数器】

```c++
void encoder_reset_counter(void)
```

功能：重置编码器计数器为 0

【关闭重置计数器】

```c++
void encoder_close_reset_counter(void)
```

功能：关闭计数器重置，恢复正常计数

4.3 信息获取函数

- `gpio_pin_t encoder_get_pulse(void)`：获取脉冲引脚号
- `gpio_pin_t encoder_get_dir(void)`：获取方向引脚号
- `gpio_mux_mode_t encoder_get_mux(void)`：获取脉冲引脚复用模式
- `enc_pwm_channel_t encoder_get_channel(void)`：获取编码器通道号

五、编码器使用示例

5.1 基本使用示例

【示例 1】单个编码器读取

```
c++
#include "lq_reg_pwm_encoder.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 初始化编码器
    // 脉冲引脚：PIN_64
    // 方向引脚：PIN_72
    ls_encoder_pwm encoder(ENC_PWM0_PIN64, PIN_72);
    printf("编码器初始化完成\n");
    printf("脉冲引脚: %d, 方向引脚: %d\n",
           encoder.encoder_get_pulse(),
encoder.encoder_get_dir());

    while(1)
    {
        // 读取编码器值
        float count = encoder.encoder_get_count();
        printf("编码器值: %.2f\n", count);
        usleep(50000);  // 50ms 延时
    }
    return 0;
}
```

【示例 2】四个编码器同时读取

```
c++
#include "lq_reg_pwm_encoder.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 初始化四个编码器
    ls_encoder_pwm enc1(ENC_PWM0_PIN64, PIN_72);
```

```
    ls_encoder_pwm enc2(ENC_PWM1_PIN65, PIN_73);
    ls_encoder_pwm enc3(ENC_PWM2_PIN66, PIN_74);
    ls_encoder_pwm enc4(ENC_PWM3_PIN67, PIN_75);

    printf("四个编码器初始化完成\n");

    while(1)
    {
        // 读取并打印四个编码器的值
        printf("编码器 1: %-6.2f  ", enc1.encoder_get_count());
        printf("编码器 2: %-6.2f  ", enc2.encoder_get_count());
        printf("编码器 3: %-6.2f  ", enc3.encoder_get_count());
        printf("编码器 4: %-6.2f\n", enc4.encoder_get_count());
        usleep(50000);  // 50ms 延时
    }
    return 0;
}
```

5.2 电机测速示例

【示例 1】电机转速测量

```
c++
#include "lq_reg_pwm_encoder.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 初始化编码器
    ls_encoder_pwm encoder(ENC_PWM0_PIN64, PIN_72);
    printf("编码器速度测量开始\n");

    while(1)
    {
        // 获取实时速度（单位：转/秒）
        float speed_rps = encoder.encoder_get_count();
        // 转换为 RPM（转/分钟）
        float speed_rpm = speed_rps * 60;
        printf("速度: %6.2f 转/秒  (%6.2f RPM)\n", speed_rps,
speed_rpm);
```

```
        usleep(50000);  // 50ms 延时
    }
    return 0;
}
```

六、编码器硬件连接

6.1 编码器信号线

典型的增量式编码器有以下信号线:

- A 相（脉冲信号）：连接到脉冲引脚（如 PIN_64）
- DIR 相（方向信号）：连接到方向引脚（如 PIN_72）
- VCC：电源正极（通常 5V 或 3.3V）
- GND：电源地

6.2 引脚分配参考

- 【编码器 1】
  - 脉冲引脚：PIN_64
  - 方向引脚：PIN_72
- 【编码器 2】
  - 脉冲引脚：PIN_65
  - 方向引脚：PIN_73
- 【编码器 3】
  - 脉冲引脚：PIN_66
  - 方向引脚：PIN_74
- 【编码器 4】
  - 脉冲引脚：PIN_67
  - 方向引脚：PIN_75

6.3 电气特性

- 输入电平：3.3V

- 输入类型：数字输入
- 最高频率：取决于 PWM 控制器时钟

注意事项：

- 1. 确保编码器输出电平与开发板兼容
- 2. 长距离传输建议使用差分信号编码器
- 3. 注意防干扰措施（屏蔽线、滤波等）

七、编码器参数配置

7.1 编码器线数配置

根据实际使用的编码器修改线数定义：

```
c++
// 在 lq_reg_pwm_encoder.hpp 中修改
#define LQ_NUM_ENCODER_LINE  (512)  // 改为实际线数
```

常见编码器线数：

- 低分辨率：100、200、256 线
- 中分辨率：512、600、1000 线
- 高分辨率：1024、2048、4096 线

八、编码器数据处理

8.1 速度单位转换

【基本单位】

encoder_get_count() 返回值单位为：转/秒

【常用单位转换】

- RPM（转/分钟）= 转每秒 × 60
- rad/s（弧度/秒）= 转每秒 × 2π

【示例代码】

```c++
float speed_rps = encoder.encoder_get_count();      // 转/秒
float speed_rpm = speed_rps * 60;                   // RPM
float speed_rad = speed_rps * 2 * 3.14159;          // rad/s
```

九、应用场景

9.1 电机速度闭环控制

适用于:

- 直流电机调速
- 速度 PID 控制
- 恒速控制

关键点:

- 选择合适的采样周期
- 实时计算速度误差
- 配合 PWM 输出控制电机

十、注意事项

- 1. 引脚配置
  - 脉冲引脚必须从预定义的枚举中选择
  - 方向引脚可以是任意 GPIO 输入引脚
  - 确保引脚不被其他功能占用
- 2. 编码器线数
  - 根据实际编码器修改线数定义
  - 线数错误会导致速度/位置计算错误
- 3. 采样频率
  - 采样频率应与编码器最高频率匹配
  - 过低会导致丢失脉冲

    - 过高会增加 CPU 负载
- 4. 信号质量
    - 确保编码器信号稳定
    - 必要时添加滤波电路
    - 避免信号线干扰
- 5. 方向引脚电平
    - 确保方向引脚有稳定电平
    - 悬空会导致方向判断错误
- 6. 多通道使用
    - 可以同时使用多个编码器
    - 每个编码器使用不同的通道

十一、常见问题

Q1: 编码器读数为 0 怎么办?

A1: 检查以下几点:

- 编码器是否正常供电
- 脉冲引脚和方向引脚连接是否正确
- 引脚复用配置是否正确
- 编码器是否在转动

Q2: 编码器读数方向相反?

A2: 可能原因:

- 方向引脚接反或电平异常
- 调换方向引脚的接线
- 或在软件中取反

Q3: 速度测量不准确?

A3: 检查以下几点:

- 编码器线数设置是否正确
- 采样周期是否合适

- 是否有脉冲丢失

Q4: 如何选择编码器线数?

A4: 选择原则:

- 高精度控制：选择高线数（1024+）
- 一般应用：中低线数（512-）
- 考虑成本和安装空间

十二、参考资源

- 龙芯 2K300/301 处理器用户手册
- 龙芯 2K300/301 开发板原理图
- 编码器示例代码:
  - example/src/05_lq_encoder_pwm_demo.cpp
- 编码器驱动代码:
  - libraries/drv/inc/lq_reg_pwm_encoder.hpp
  - libraries/drv/src/lq_reg_pwm_encoder.cpp